

RAR: Per-Flow Reconstruction-Aware Routing for Encrypted Traffic Class-Incremental Learning

Anonymous Author(s)

Abstract

Encrypted traffic classification identifies applications and services without inspecting packet payloads, supporting network security, service management, and traffic measurement. Modern deep representations perform strongly on static, closed-set traffic benchmarks, yet class-incremental encrypted traffic classification remains challenging when new applications continually emerge and historical data cannot be fully retained. Flows from the same application can vary substantially, while flows from different applications can appear similar because they share protocols, cloud services, or third-party components. Consequently, the relevance of historical knowledge varies across flows, whereas existing replay and distillation methods typically use fixed global weights that cannot adapt at the flow level. We find that the reconstruction error of an autoencoder trained on historical traffic provides a per-flow signal of compatibility with historical class manifolds. Based on this finding, we propose Reconstruction-Aware Routing (RAR). For each flow, RAR converts its reconstruction error into a continuous old/new compatibility score and uses the same score to control both historical-knowledge preservation and new-knowledge acquisition during training, as well as the soft integration of a frozen historical-class path and a trainable new-class path during inference. Further logit calibration enables joint prediction over a unified traffic-class space. Experiments on six public datasets show that RAR reduces forgetting by 35.5%–67.5% relative to the strongest evaluated classical CIL baseline and improves mixed-class accuracy by up to 11.7 percentage points. Multi-step and replay-budget experiments further support reconstruction-conditioned old/new balancing.

CCS Concepts

• Security and privacy → Network security; • Computing methodologies → Transfer learning; • Networks → Traffic classification.

Keywords

Encrypted Traffic Classification, Class-Incremental Learning, Knowledge Distillation, Reconstruction-Aware Routing

ACM Reference Format:

Anonymous Author(s). 2026. RAR: Per-Flow Reconstruction-Aware Routing for Encrypted Traffic Class-Incremental Learning. In . ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Encrypted-traffic classifiers identify applications and services from packet metadata, length distributions, and timing patterns alone, and the set of classes they must recognize grows continuously. New applications launch, protocol implementations change, and each addition forces the classifier to recognize a class it has never seen. Operators run these classifiers under tight latency budgets and cannot retrain from scratch on every addition; the retraining cost grows with the class history.

Class-incremental learning (CIL) targets this need by adding classes without retraining on the full history. Classical CIL methods use distillation and replay to preserve old knowledge, as in LwF [7], iCaRL [10], and DER [12]. Applied to encrypted traffic, three structural gaps remain. First, expanding the classification head makes old and new classes compete for one shared decision boundary. Second, these methods generally balance preservation and acquisition at the dataset or class level rather than adapting the balance to each input. Third, replay depends on a limited buffer that cannot represent the full old-class distribution.

We observe that an autoencoder fitted to frozen features of replayed old samples tends to reconstruct old-class samples with lower error than new-class samples. In encrypted-traffic representations, reconstruction error therefore provides a per-flow score of compatibility with the replay-derived historical-class manifold. Reconstruction-based task gates have used related signals to select task-specific experts [1], while OOD-based CIL methods infer a task before applying a corresponding classifier [4–6, 8]. RAR instead uses compatibility as a continuous control signal for balancing old and new traffic classes within one prediction space.

RAR (Reconstruction-Aware Routing) is a reconstruction-conditioned CIL framework for encrypted-traffic classification. It turns the compatibility score into a soft routing weight over a frozen old-class path inherited from the preceding step and a trainable path for the arriving classes. The same score weights the distillation and margin objectives, coupling incremental optimization with the inference rule. A logit calibration layer then makes the two class groups comparable in a joint prediction over all seen traffic classes. A reconstruction temperature τ controls routing softness: small τ approaches deterministic path selection; large τ smoothly fuses both paths.

We make three contributions.

- (1) **Reconstruction-conditioned encrypted-traffic CIL.** We formulate compatibility with a replay-derived historical traffic manifold as a continuous per-flow signal for balancing previously learned and newly arriving traffic classes.
- (2) **Calibrated dual-path integration.** We use the same compatibility signal to weight the old-class distillation loss and new-class cross-path margin during training, and to softly

route logits between frozen old-class and trainable new-class paths during inference, enabling joint prediction over all seen classes without task identifiers.

- (3) **Evaluation across encrypted-traffic domains.** Across six public datasets, RAR reduces forgetting by 35.5%–67.5% and improves mixed-class accuracy by up to 11.7 percentage points over the evaluated classical CIL baselines. Additional studies examine backbone choice, incremental steps, replay budgets, and component contributions.

The remainder of this paper evaluates the contribution. §2 formalizes the encrypted-traffic CIL problem and its three structural gaps. §3 presents RAR’s mechanism, from reconstruction error through soft routing to calibration, and closes with a training algorithm (Algorithm 1). §4 reports empirical evidence on six datasets: RAR against classical CIL strategies on ET-BERT, a backbone study confirming that naive forgetting persists across nine encoders, a component ablation, and robustness under multi-step increments and tight replay budgets. §5 positions RAR against prior work. §6 summarizes the contributions, limitations, and future directions.

2 Preliminaries

2.1 The Encrypted-Traffic CIL Setup

We formalize the problem as class-incremental learning (CIL). At the base step, we train on K_0 classes. At each incremental step, K_i new classes arrive with a small replay buffer of old samples.

At the incremental step T_i the classifier must satisfy two competing objectives. It must acquire the new classes with high accuracy, and it must retain the old classes despite the head-expansion operation that adds K_i output dimensions to the classification layer. The replay buffer typically holds a fixed small number of samples per old class, which is the only bridge between the base step and the incremental step.

Encrypted-traffic classification adds three domain-specific pressures on top of generic CIL. Packet payloads are encrypted and uninformative; the classifier must rely on packet length, timing, direction, and TLS handshake features. Class boundaries shift as new applications and services appear and protocol behaviors evolve. And the deployed classifier must run online under tight latency budgets, so retraining on the full history on every addition is rarely practical.

2.2 The ET-BERT Backbone

RAR adopts ET-BERT [9] as its frozen encoder. ET-BERT is a Transformer pre-trained on large-scale unlabeled encrypted traffic through two self-supervised tasks: a Masked BURST Model that captures byte-level correlations within transmission bursts, and a Same-origin BURST Prediction that models sequential relationships across consecutive bursts. The pre-trained tokenizer converts raw packet sequences into fixed-length input tokens, and the encoder produces class-separable features that transfer across encrypted-traffic domains without per-task labels. We freeze the encoder at its pre-trained weights and build the routing mechanism on top of its representations; the frozen encoder is shared by both classification heads and the reconstruction autoencoder.

2.3 Three Structural Gaps

Three structural gaps in the classical CIL families evaluated in this paper help explain why forgetting persists on encrypted-traffic classification; closing any one of them in isolation is insufficient.

Shared classifier competition. Expanding the classification head to accommodate new classes forces old and new outputs to share one decision space [10, 11]. Under class imbalance, the shared cross-entropy objective can favor the group with larger or more recent gradients and shift predictions away from old classes. Replay can mitigate this effect, but its effectiveness depends on the size and representativeness of the buffer.

Knowledge-unaware distillation. Distillation-based CIL methods (LwF [7], iCaRL [10]) use a globally weighted preservation objective. The fixed weight cannot adapt to how compatible each flow is with previously learned class representations: flows close to and far from the historical-class manifold are regulated in the same way. Distillation can therefore preserve average historical behavior while overlooking flow-level differences in the required stability–plasticity balance.

Limited replay representation. Replay-based CIL methods (DER [12], iCaRL [10]) depend on a small buffer of stored old samples that cannot represent the full old-class distribution. Below a buffer-size threshold the replay buffer stores only a few local modes of each old class, and the autoencoder or replayed features that depend on the buffer inherit the under-representation. The forgetting rate therefore degrades sharply below a buffer-size threshold that the deployment context often cannot meet.

3 Method

RAR’s design follows a single line: per-flow historical-class compatibility, reconstruction-aware soft routing, dual class paths, and calibration. A reconstruction autoencoder over frozen traffic representations produces an old/new compatibility score for each flow. During training, the score weights the old-class distillation loss and new-class cross-path margin; during inference, it integrates a frozen historical-class path with a trainable new-class path. Logit calibration then maps the two outputs into a unified traffic-class space. The rest of this section defines each component and the assumptions that bound the mechanism.

3.1 Problem Formulation

Let $f : \mathcal{X} \rightarrow \mathbb{R}^d$ be a frozen encoder pre-trained on large-scale encrypted traffic data (ET-BERT [9]). At the base stage, we train a classifier F_0 on K_0 classes using $\mathcal{D}_0 = \{(x_i, y_i)\}_{i=1}^{N_0}$. At increment t , K_t classes arrive with data $\mathcal{D}_t$, and a replay buffer $\mathcal{B}_{t-1}$ represents the classes seen before t . Let $K_{<t} = \sum_{j=0}^{t-1} K_j$ denote the number of previously learned classes. RAR freezes the complete preceding classifier F_{t-1} as the old prediction path and learns a new-class head $h_t : \mathbb{R}^d \rightarrow \mathbb{R}^{K_t}$. For the single-increment setting, $F_{t-1} = F_0$ reduces to the frozen base classifier. Each flow is routed between F_{t-1} and h_t by a weight derived from reconstruction error. The following subsections define the reconstruction signal (§3.2), routing (§3.3), calibration (§3.4), and training procedure (§3.5).

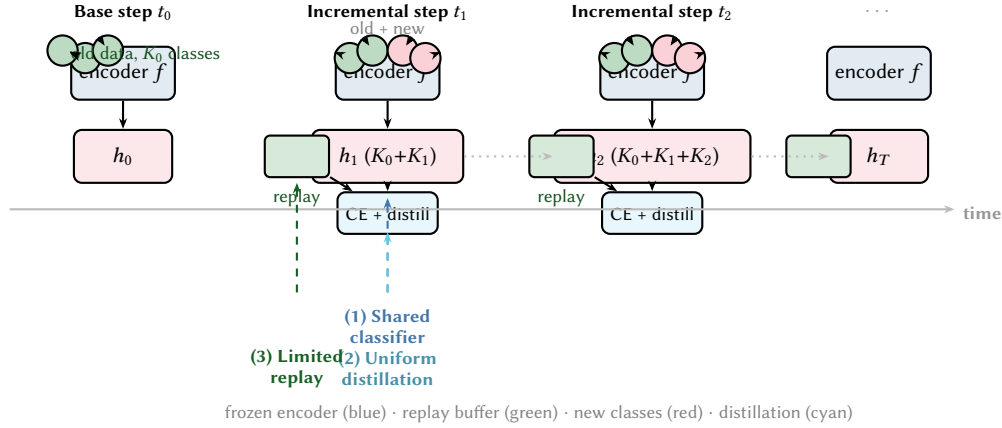


Figure 1: Class-incremental learning setup in encrypted-traffic classification.

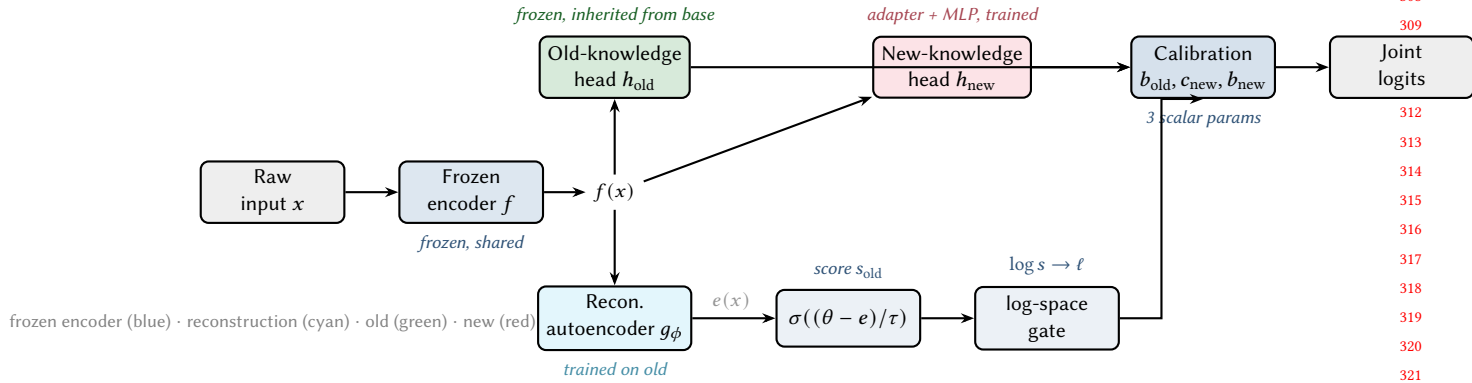


Figure 2: RAR architecture overview.

3.2 Per-Flow Reconstruction Compatibility

RAR measures each flow’s compatibility with the replay-derived historical-class manifold using a scalar routing score. This score is an operational weight, not a calibrated posterior probability or a hard assignment to one class group.

We train a reconstruction autoencoder g_ϕ on frozen-encoder features of replayed old samples, so g_ϕ learns the manifold that the frozen encoder f produces for old classes. The reconstruction error therefore tends to be small on old-class samples and large on new-class samples:

$$e(x) = \frac{1}{d} \|g_\phi(f(x)) - f(x)\|_2^2. \quad (1)$$

We compute the old 95th-percentile edge and new 5th-percentile edge on the replay and new reference sets, respectively, and set θ to their midpoint. The temperature τ is the pooled standard deviation of their errors, lower-bounded for numerical stability. We then

convert the error to per-flow routing scores:

$$s_{old}(x) = \sigma\left(\frac{\theta - e(x)}{\tau}\right), \quad (2)$$

$$s_{new}(x) = 1 - s_{old}(x), \quad (3)$$

where τ controls the sharpness of the sigmoid. The fitted threshold and temperature are fixed after the pre-training stage; only $e(x)$ is computed per input at inference.

The score does not require a class label at inference: it measures compatibility between $f(x)$ and the feature manifold learned by g_ϕ . Its fitting stage does use the known old/new split between replay and newly arriving reference samples. Reconstruction-based gating has previously been used to infer tasks and select experts [1]; here, the score instead balances two class groups within a unified prediction space and weights the corresponding incremental-learning objectives. It complements output-based distillation (LwF [7], iCaRL [10]) and stored-example replay (DER [12]).

The empirical foundation for s_{old} is separability: across the six evaluated datasets, reconstruction error distinguishes old from new samples with mean AUC 0.894 (Figure 4). This result motivates

evaluating the routing mechanism on the full benchmark; it does not imply perfect old/new identification.

3.3 Soft Routing and Dual Class Paths

RAR routes each flow through two class paths: the frozen preceding classifier F_{t-1} and a trainable new-class head h_t equipped with a lightweight adapter. Their outputs are weighted by $s_{\text{old}}(x)$ and $s_{\text{new}}(x)$ from Eqs. (2)–(3).

In the first increment, the old path is the frozen base-stage classifier. In later increments, it is the complete recursively constructed RAR classifier from the preceding step and therefore covers all previously seen classes. The current new head is trained through a residual adapter and a LayerNorm+MLP pathway. The current encoder remains frozen.

We apply the routing signal in log space. Let $\ell_{\text{old}}(x) \in \mathbb{R}^{K_{<t}}$ and $\ell_{\text{new}}(x) \in \mathbb{R}^{K_t}$ be the raw logit vectors from the two paths. The gated logits are computed by adding the log of each path's routing score:

$$\tilde{\ell}_{\text{old}}(x) = \ell_{\text{old}}(x) + \log s_{\text{old}}(x), \quad (4)$$

$$\tilde{\ell}_{\text{new}}(x) = \ell_{\text{new}}(x) + \log s_{\text{new}}(x). \quad (5)$$

The two gated vectors are concatenated into a joint prediction over all $K_{<t} + K_t$ classes:

$$\hat{y} = \text{softmax}([\tilde{\ell}_{\text{old}}(x); \tilde{\ell}_{\text{new}}(x)]). \quad (6)$$

In softmax space, adding log s is equivalent to multiplying the head's class probabilities by s : the old-head probabilities are scaled by $s_{\text{old}}(x)$ and the new-head probabilities by $s_{\text{new}}(x)$, so both heads contribute to every prediction according to the flow's historical-class compatibility. As $\tau \rightarrow 0$ the sigmoid sharpens toward a step function and s_{old} approaches 0 or 1; as $\tau \rightarrow \infty$ the sigmoid flattens and both heads contribute near-equally. §4.4 reports the empirical trade-off.

We separate old and new paths because standard head expansion makes their logits compete within one trainable decision space [10, 11]. Under imbalanced replay, optimization can favor the newly arriving group. Freezing the preceding path prevents direct updates to its class-specific parameters, while the calibration step makes its logits comparable with those of the new head. Table 5 measures the contribution of this separation.

We use soft routing because hard assignment routes every sample to a single head and discards the second head's view entirely. On samples near the reconstruction-error decision boundary, the discarded head often holds the correct prediction. Soft routing keeps both views available and shifts their contribution with the input, so the calibration step is structurally necessary: the two heads cannot be averaged directly because they output logits over disjoint class subsets.

3.4 Logit Calibration

Each head outputs logits over its own class subset; a calibration layer removes the scale mismatch between the two paths before their logits are combined.

The two heads live on different class axes and may produce logits at different magnitudes. A naive concatenation would let the head with larger logits dominate Eq. (6) regardless of the routing scores.

Calibration equalizes the two heads' output scales on a held-out validation set before the logits are combined.

We learn three scalar parameters: a subtractive bias $b_{\text{old}} \in \mathbb{R}$ on the old-path logits, a multiplicative scale $c_{\text{new}} \in \mathbb{R}^+$, and an additive bias $b_{\text{new}} \in \mathbb{R}$ on the new-head logits:

$$\ell'_{\text{old}} = \tilde{\ell}_{\text{old}} - b_{\text{old}}, \quad (7)$$

$$\ell'_{\text{new}} = c_{\text{new}} \cdot \tilde{\ell}_{\text{new}} + b_{\text{new}}, \quad (8)$$

where $\tilde{\ell}_{\text{old}}, \tilde{\ell}_{\text{new}}$ are the gated logits from Eqs. (4)–(5). The concatenation in Eq. (6) then uses $[\ell'_{\text{old}}; \ell'_{\text{new}}]$. The three scalars are optimized with the new-class pathway during incremental training and then refined, with both prediction paths frozen, on a balanced calibration set formed from replayed old samples and held-out new samples.

Calibration does not change which classes the paths cover; it only adjusts their relative output scales. The classification decision remains a weighted combination in which the weights come from the reconstruction-derived routing score. Table 5 reports its contribution.

3.5 Training Objective

RAR training proceeds in three phases at each increment.

Phase 1: Fitting the reconstructor. We fit the reconstruction autoencoder g_ϕ on frozen-encoder features of replayed old samples by minimizing $\mathcal{L}_{\text{recon}} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \|g_\phi(f(x)) - f(x)\|_2^2$. We then estimate θ and τ from replayed-old and held-out-new errors as described in §3.2, and freeze g_ϕ .

Phase 2: Incremental training. We freeze the preceding prediction path and optimize the adapter, new-class pathway, and calibration scalars on the mixed incremental training set $\mathcal{S}_t = \mathcal{B}_{t-1} \cup \mathcal{D}_t$ with the following composite loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \mathcal{L}_{\text{distill}} + \beta \mathcal{L}_{\text{local}} + \gamma \mathcal{L}_{\text{margin}} + \delta \mathcal{L}_{\text{reg}}, \quad (9)$$

$$\mathcal{L}_{\text{distill}} = \frac{1}{Z} \sum_{x \in \mathcal{S}_t} s_{\text{old}}(x) D_{\text{KL}}(F_{t-1}(x) \parallel \ell_{\text{old}}(x)). \quad (10)$$

Here $Z = \sum_{x \in \mathcal{S}_t} s_{\text{old}}(x)$ normalizes the per-flow weights. ℓ_{old} denotes the preceding path's logits after the trainable old-path bias, so the KL term anchors this adjustment to the frozen teacher output; the class-specific parameters of F_{t-1} remain frozen. $\mathcal{L}_{\text{CE}}$ is cross-entropy on the concatenated output over all classes; $\mathcal{L}_{\text{local}}$ is cross-entropy restricted to the new head on new-class samples; $\mathcal{L}_{\text{margin}}$ penalizes the old path for outscoring the correct new-class logit, weighted by $s_{\text{new}}(x)$; and $\mathcal{L}_{\text{reg}} = \|a(x) - f(x)\|_2^2$ regularizes the adapter shift from the frozen features. The replay buffer uses 1% of the old training data in the main comparison; §4.5 sweeps this ratio.

Phase 3: Calibration refinement. After incremental training, we freeze both prediction paths and refine $(b_{\text{old}}, c_{\text{new}}, b_{\text{new}})$ on a balanced mixed calibration set by minimizing cross-entropy. The old portion is sampled from replay memory; the new portion comes from a fixed new-class validation partition that is excluded from final testing.

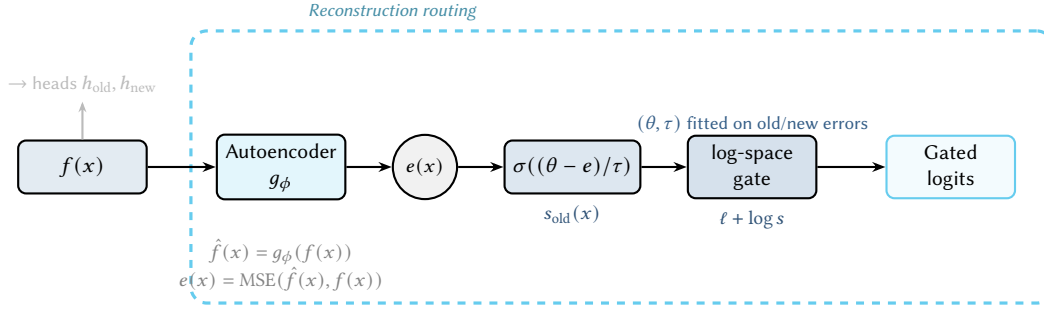


Figure 3: Reconstruction routing mechanism.

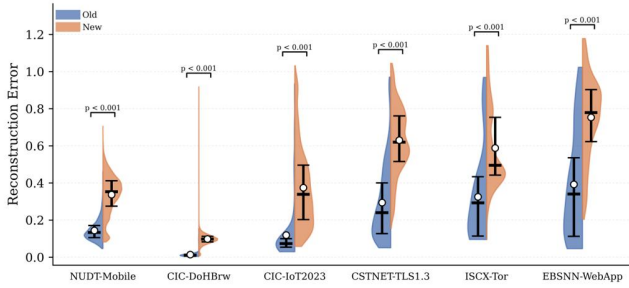


Figure 4: Per-dataset reconstruction-error distributions (old vs. new).

3.6 Recursive Multi-step Update

For sequential increments, RAR applies the same construction recursively. After increment t , the complete calibrated classifier F_t , including its earlier routing hierarchy, is frozen and used as the old path at increment $t + 1$. The replay buffer is resampled from all classes seen through t , and a fresh reconstructor, threshold, temperature, adapter, and new-class head are fitted for the next class block. Consequently, the old output dimension grows from $\sum_{j=0}^{t-1} K_j$ to $\sum_{j=0}^t K_j$ without flattening the previous heads into a newly trainable shared classifier. This is the protocol used in the five-increment experiment in §4.5.

Algorithm. Algorithm 1 summarizes the full training and inference procedure.

The temperature τ trades routing determinism against path fusion. In the implementation it is estimated from the pooled reconstruction-error spread; §4.4 additionally reports a controlled sensitivity sweep. Lowering τ makes the routing weights more discrete, but does not by itself reduce computation because both paths are still evaluated.

3.7 Applicability and Assumptions

RAR is designed under two key assumptions that define its applicability.

First, replay memory should provide representative coverage of previously learned classes. If replay samples fail to capture the historical-class distribution, the reconstructor may learn an incomplete historical manifold, leading to inaccurate routing scores. §4.5

Algorithm 1: Reconstruction-Aware Routing (RAR)

Input: Frozen preceding classifier F_{t-1} ; replay buffer $\mathcal{B}_{t-1}$; current data $\mathcal{D}_t$; reconstructor g_ϕ ; temperature τ
Output: Incremental classifier F

- 1 **for each batch from $\mathcal{B} \cup \mathcal{D}_{\text{new}}$ do**
- 2 **Feature extraction;**
- 3 $z \leftarrow f(x)$ (frozen, no gradient);
- 4 **Reconstruction error;**
- 5 $e(x) = \frac{1}{d} \|g_\phi(z) - z\|_2^2$;
- 6 **Old/new routing score;**
- 7 $s_{\text{old}} = \sigma\left(\frac{\theta - e(x)}{\tau}\right)$;
- 8 $s_{\text{new}} = 1 - s_{\text{old}}$;
- 9 **Dual-head routing (log-space);**
- 10 $\tilde{\ell}_{\text{old}} = \ell_{\text{old}} + \log s_{\text{old}}$;
- 11 $\tilde{\ell}_{\text{new}} = \ell_{\text{new}} + \log s_{\text{new}}$;
- 12 $\hat{y} = \text{softmax}([\tilde{\ell}_{\text{old}}; \tilde{\ell}_{\text{new}}])$;
- 13 **Training objective;**
- 14 $\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \mathcal{L}_{\text{distill}} + \beta \mathcal{L}_{\text{local}} + \gamma \mathcal{L}_{\text{margin}} + \delta \mathcal{L}_{\text{reg}}$;
- 15 **end**
- 16 **Post-hoc calibration;**
- 17 $\ell'_{\text{old}} = \tilde{\ell}_{\text{old}} - b_{\text{old}}$;
- 18 $\ell'_{\text{new}} = c_{\text{new}} \cdot \tilde{\ell}_{\text{new}} + b_{\text{new}}$;
- 19 Refine $(b_{\text{old}}, c_{\text{new}}, b_{\text{new}})$ on the mixed calibration set (paths frozen);
- 20 **return F**

evaluates different replay buffer sizes and shows that RAR remains robust even under a memory ratio of 0.005.

Second, reconstruction error should exhibit sufficient separability between old and emerging knowledge. When old and new classes have highly similar feature representations, reconstruction errors may overlap, reducing the reliability of the compatibility score. Figure 4 shows the observed separation across the six evaluated datasets.

These assumptions characterize the applicability of RAR rather than strict limitations. RAR is most effective when replay samples preserve representative historical-class coverage and when old and emerging classes exhibit distinguishable representation patterns.

Table 1: Six encrypted-traffic datasets used in evaluation.

Dataset	Domain	Public	#Cls	#Flows
NUDT-Mobile	Mobile app	Yes	300	496086
CIC-DoHBrw	DNS-over-HTTPS	Yes	6	49777
CIC-IoT2023	IoT attack	Yes	22	26413
CSTNET-TLS1.3	TLS 1.3 app	Yes	120	43243
ISCX-Tor	Tor	Yes	26	8518
EBSNN-WebApp	Web app	Yes	48	15336

Table 2: Baseline groups.

Group	Variable held fixed	Baselines
A	ET-BERT encoder	LwF, iCaRL, BiC, DER, Adapter, Fine-tune, MORE
B	Replay + head expansion	AppScanner, ETC-PS, FlowLens, CNN-based, CNN-emb, EBSNN, FS-Net, GraphDApp, MM4flow

4 Experiments

4.1 Experimental Setup

We evaluate RAR on six public encrypted-traffic datasets that span several application and threat surfaces: mobile app traffic (NUDT-Mobile), DNS-over-HTTPS (CIC-DoHBrw), IoT attack traffic (CIC-IoT2023), TLS 1.3 application traffic (CSTNET-TLS1.3), Tor traffic (ISCX-Tor), and web application traffic (EBSNN-WebApp). Table 1 reports their sizes and class counts.

We compare RAR against two baseline groups. *Group A* holds the encoder fixed at ET-BERT [9] and varies the CIL strategy: Learning without Forgetting (LwF) [7], iCaRL [10], BiC [11], DER [12], Adapter [3], Fine-tune, and an ET-BERT adaptation of MORE [5]. *Group B* holds the naive CIL setup fixed and varies the traffic-classification backbone across nine models: AppScanner, ETC-PS, FlowLens, CNN-based, CNN-emb, EBSNN, FS-Net, GraphDApp, and MM4flow. RAR with ET-BERT is included as a reference rather than counted as a tenth backbone baseline. Group A compares routing with both classical CIL strategies and a closely related OOD-replay multi-head method on a fixed encoder; Group B tests whether naive incremental forgetting persists across encoder choices.

Five metrics isolate the quantity each claim depends on. *old-before* measures accuracy on old classes after head expansion and *before* training on the new step. *old-after* measures accuracy on old classes after training on the new step. *new-after-train* measures accuracy on the new classes after the new step. *mix* reports combined accuracy on old plus new classes after the new step. *forgetting* equals *old-before* minus *old-after* and isolates training-induced forgetting, so the head-expansion drop alone does not contribute.

We report forgetting and mix as the headline pair: forgetting measures old-class retention after incremental training, and mix captures the joint old/new accuracy that operators observe. Together they evaluate the two outcomes that reconstruction-conditioned balancing is designed to improve.

All experiments share one training and evaluation pipeline. The encoder remains frozen. During an increment, the inherited old path

and the fitted reconstructor are frozen; the adapter, new-class pathway, and calibration scalars are trainable. The calibration scalars are subsequently refined on a mixed validation set.

Before incremental training, we repartition the provided new-class evaluation split using a fixed random seed (42), reserving 10% for validation and the remaining 90% for held-out testing. The validation and held-out test subsets are disjoint, and the old-class test split is evaluated independently.

The main comparison uses a two-stage protocol: a base stage trains the initial head on the first half of classes, and one incremental stage learns the remaining classes with a fixed replay buffer. Section 4.5 additionally evaluates the recursive five-increment protocol defined in Section 3.6.

4.2 Comparison with CIL Strategies

Against classical CIL strategies on a fixed ET-BERT encoder, RAR cuts forgetting by an average of 47.8% (range 35.5%–67.5%) across six datasets and raises mixed-class accuracy by up to 11.7 percentage points over the strongest evaluated baseline (Adapter).

Replay-based methods (DER, iCaRL) outperform uniform distillation (LwF) under encrypted traffic, yet both still forget more than RAR on every dataset.

Fine-tune, which omits all anti-forgetting mechanisms, provides a naive incremental reference. RAR reduces forgetting by 35.5%–67.5% relative to the strongest evaluated baseline (Adapter) on every dataset.

The consistent gains are compatible with the intended effect of reconstruction-error routing: adapting preservation and acquisition signals to each flow rather than applying a uniform balance.

4.3 Backbone Study

On NUDT-Mobile, naive incremental training forgets across all nine traffic-classification backbones, which isolates the forgetting problem from the encoder choice.

Forgetting spans 13.3%–58.4% across the nine naive backbones despite their different representation architectures. Even the lowest-forgetting naive backbone (GraphDApp, 13.3%) forgets more than twice as much as RAR’s 6.65% under the evaluated setup.

Changing the encoder alone does not eliminate the old-class accuracy loss under naive incremental training. RAR yields lower forgetting than every naive-backbone configuration in this study (6.65% vs 13.3% for the lowest-forgetting configuration, GraphDApp).

Table 4 reports the naive-incremental forgetting rate and mix accuracy for each of the nine backbones on NUDT-Mobile, with RAR shown for contrast.

4.4 Ablation Study

Each of RAR’s three design choices contributes a measurable share of the overall gain; removing any one reopens part of the forgetting gap. We present the NUDT-Mobile results in the main text, and the same pattern holds across all six datasets.

Table 5 reports component-level ablations on NUDT-Mobile. Removing the reconstruction routing signal raises forgetting by 5.78 pp; reverting to a single head raises forgetting by 4.49 pp and drops mix by 9.77 pp; removing calibration raises forgetting by

Table 3: Forgetting ($F \downarrow$, lower is better) and mixed-class accuracy ($M \uparrow$, higher is better). Mean over 3 seeds; *TBD* marks runs in progress.

Method	Dataset											
	NUDT		DoH		IoT		TLS		Tor		Web	
	$F \downarrow$	$M \uparrow$	$F \downarrow$	$M \uparrow$	$F \downarrow$	$M \uparrow$	$F \downarrow$	$M \uparrow$	$F \downarrow$	$M \uparrow$	$F \downarrow$	$M \uparrow$
Fine-tune	95.40	33.71	91.58	36.74	88.52	11.17	90.99	46.52	86.25	41.31	86.01	43.19
LwF [7]	61.03	55.91	88.91	36.75	52.58	43.53	56.21	65.66	96.37	34.72	78.11	48.20
iCaRL [10]	25.15	81.66	33.87	73.25	12.39	79.47	28.87	80.83	73.60	69.90	37.77	73.97
BiC [11]	24.78	79.48	33.87	73.25	11.68	80.09	28.94	80.65	84.87	42.21	38.15	73.61
DER [12]	12.64	86.15	33.92	73.19	11.33	79.37	16.85	85.11	77.28	46.83	17.89	84.78
Adapter [3]	12.79	86.86	33.86	73.26	9.22	82.12	16.40	87.20	13.11	89.16	13.98	88.69
MORE [5]	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
RAR (ours)	6.65	90.12	11.00	84.93	4.82	86.05	8.49	91.70	8.45	91.94	8.37	92.30

Table 4: Naive-incremental forgetting and mix (%) on NUDT-Mobile.

Backbone	Forgetting (%)	Mix (%)
AppScanner	58.37	49.30
ETC-PS	56.63	47.99
FlowLens	56.99	45.69
CNN-based	44.10	58.66
CNN-emb	41.27	63.89
EBSNN	30.81	73.37
FS-Net	23.35	74.70
GraphDApp	13.26	55.54
MM4flow	22.45	81.61
RAR (ours)	6.65	90.12

Table 5: Ablation results on NUDT-Mobile; *TBD* marks runs in progress.

Variant	Forgetting (%)	Mix (%)
w/o routing	12.43	87.29
constant routing	<i>TBD</i>	<i>TBD</i>
w/o dual-head	11.14	80.35
w/o calibration	10.06	87.82
RAR (full)	6.65	90.12

3.41 pp. These completed results quantify the effect of each component as implemented in the complete framework; the routing ablation removes both loss weighting and logit gating and therefore does not attribute the gain to either use in isolation. The constant-routing control retains both uses but replaces the per-flow score with $s_{old} = s_{new} = 0.5$; its run is pending in this draft.

Reconstruction temperature τ (design knob). Figure 5 reports forgetting and mix on CSTNET-TLS1.3 as τ sweeps from 0.25 (near-deterministic head selection) to 4.0 (near-uniform fusion). Forgetting drops from 9.71% at $\tau=0.25$ to 7.78% at $\tau=4.0$, and mix rises from 91.03% to 92.06%. The curve flattens after $\tau=1.0$: the difference

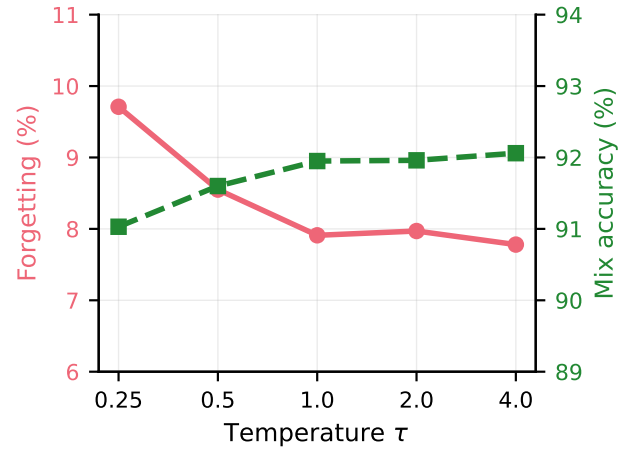


Figure 5: Forgetting and mix as a function of τ , on CSTNET-TLS1.3.

between $\tau=1.0$ and $\tau=4.0$ is under 0.2 pp on both metrics. RAR therefore exposes a tunable knob that converges quickly; the operator can fix $\tau=1.0$ (the fitted value) without further tuning.

4.5 Robustness

Over T incremental steps and a range of replay-buffer sizes, RAR maintains its lead over Adapter, the strongest evaluated classical CIL baseline overall.

We select Adapter for these robustness experiments because it provides the strongest overall performance among the evaluated classical CIL baselines in Table 3, while allowing the same frozen ET-BERT encoder, data splits, replay protocol, and memory budgets as RAR. This controlled comparison tests whether RAR’s advantage persists across incremental steps and constrained replay budgets; the broader method comparison remains in Table 3.

We split the 40 new classes of CSTNET-TLS1.3 into $T = 5$ steps of 8 classes each. Fine-tune collapses from 75.6% to 22.0% mix accuracy after five steps. Adapter degrades from 91.3% to 74.8%. RAR drops

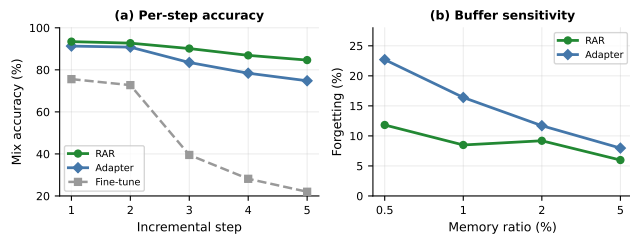


Figure 6: Robustness on CSTNET-TLS1.3: (a) per-step mix accuracy over $T = 5$ steps, and (b) forgetting as a function of memory ratio.

only from 93.4% to 84.6%, widening its lead over Adapter from 2.1 pp at $T=1$ to 9.8 pp at $T=5$.

Across all evaluated memory ratios, RAR exhibits lower forgetting than Adapter. At the smallest ratio of 0.005, RAR records 11.82% forgetting, compared with 22.70% for Adapter. Increasing the ratio to 0.05 reduces the endpoint values to 5.98% and 7.97%, respectively, although the intermediate RAR results are not strictly monotonic. Notably, RAR at ratio 0.005 achieves forgetting comparable to Adapter at a four-times-larger ratio of 0.02 (11.82% versus 11.70%).

Figure 6 reports the per-step accuracy and buffer sensitivity on CSTNET-TLS1.3.

5 Related Work

5.1 Class-Incremental Learning

Classical CIL methods commonly combine distillation, calibration, and replay. The representative methods considered here generally use class-level or dataset-level balancing rather than an input-dependent old/new routing score.

Distillation-based methods (LwF [7], iCaRL [10]) match the soft outputs of the old model using a globally weighted objective. The structural limitation is that a fixed coefficient does not adapt the preservation pressure to each sample’s compatibility with historical classes.

Calibration-based methods such as BiC [11] learn a linear correction for the new-class logits after incremental training. This correction is shared across inputs and therefore addresses systematic old/new bias rather than input-dependent routing.

Replay-based methods (DER [12]) store old samples or features and replay them during incremental training. The structural limitation is the buffer: a fixed buffer cannot represent the full old-class distribution, and the model’s behavior on unstored samples inherits the buffer’s under-representation.

RAR adds a per-flow old/new compatibility score derived from reconstruction error. The score gates the two class paths and weights the distillation and margin losses, while replay supplies the historical samples used to fit the reconstructor and train the incremental model.

5.2 Task Inference and Expert Routing

Input-dependent routing predates RAR. Expert Gate [1] trains an autoencoder for each sequentially learned task and selects the expert whose autoencoder yields the lowest reconstruction error for a test sample. It establishes reconstruction error as a task-inference signal. A complementary line connects CIL to task prediction and OOD detection: CLOM [4] trains task-specific OOD models, a theoretical analysis formalizes within-task and task-ID prediction as the two components of CIL [6], MORE [5] builds task-specific heads with OOD replay, and TPL [8] improves task prediction through likelihood ratios.

RAR does not claim reconstruction-based task inference or multi-head CIL as new. It addresses encrypted-traffic CIL with a different integration: a single reconstructor fitted to replayed historical traffic produces a continuous per-flow old/new compatibility score; the score weights the old-class distillation loss and new-class cross-path margin and softly gates two disjoint class groups; calibration then supports joint prediction over all seen traffic classes. The contribution is this reconstruction-conditioned training and inference framework, rather than any component in isolation.

5.3 Traffic Classification Backbones

Encrypted-traffic classification research has produced a series of strong backbones (ET-BERT [9], AppScanner, ETC-PS, FlowLens, CNN-based, CNN-emb, EBSNN, FS-Net, GraphDApp, MM4flow) that pre-train on packet-level features and reach high accuracy on static label sets. The structural limitation of this line of work is that it solves representation learning, not incremental learning: a stronger backbone still forgets old classes when its classification head expands to accommodate new ones. We quantify this in §4.3: naive incremental training forgets across all nine backbones on NUDT-Mobile, and RAR (on top of ET-BERT) breaks out of the band.

RAR’s reconstruction-error routing operates on frozen encoder features and therefore depends on their separability. The routing mechanism does not require the encoder itself to be incremental-aware, and its per-flow score can be incorporated into frozen-encoder CIL pipelines. Recent work on incremental representation learning, such as MEMENTO [2], instead adapts the backbone to the class stream. RAR studies the complementary routing problem; combining the two forms of adaptation remains future work.

6 Conclusion

RAR presents a reconstruction-conditioned CIL framework for encrypted-traffic classification. It transforms per-flow reconstruction errors into historical-class compatibility scores and uses the same signal to weight the old-class distillation loss and new-class cross-path margin during training and to softly route predictions between old- and new-class paths during inference. The contribution lies in this traffic-specific integration for unified-label-space CIL rather than in reconstruction-based task inference itself. Experiments on six encrypted-traffic datasets show that RAR consistently reduces forgetting while improving mixed-class recognition performance over the evaluated classical CIL baselines.

Although effective, RAR still relies on limited replay samples to characterize the historical-class manifold and may face challenges

when old and new classes exhibit highly similar representations. Future work will explore more adaptive compatibility signals and integrate reconstruction-aware routing with incremental representation learning to further enhance continual traffic classification.

References

- [1] Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. 2017. Expert Gate: Lifelong Learning with a Network of Experts. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 3366–3375.
- [2] Anonymous. 2024. MEMENTO: Incremental Representation Learning for Encrypted Traffic. In *Proceedings of the ACM Conference*. MEMENTO placeholder – obtain full citation and replace before submission..
- [3] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. Adapter baseline placeholder – replace before submission..
- [4] Gyuhak Kim, Sepideh Esmailpour, Changnan Xiao, and Bing Liu. 2022. Continual Learning Based on OOD Detection and Task Masking. *arXiv preprint arXiv:2203.09450* (2022).
- [5] Gyuhak Kim, Zixuan Ke, and Bing Liu. 2022. A Multi-Head Model for Continual Learning via Out-of-Distribution Replay. *arXiv preprint arXiv:2208.09734* (2022).
- [6] Gyuhak Kim, Changnan Xiao, Tatsuya Konishi, Zixuan Ke, and Bing Liu. 2022. A Theoretical Study on Solving Continual Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [7] Zhizhong Li and Derek Hoiem. 2017. Learning without Forgetting. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Placeholder entry – replace before submission..
- [8] Haowei Lin, Yijia Shao, Weinan Qian, Ningxin Pan, Yiduo Guo, and Bing Liu. 2024. Class Incremental Learning via Likelihood Ratio Based Task Prediction. In *International Conference on Learning Representations (ICLR)*.
- [9] Xinjie Lin, Gang Xiong, Gaopeng Gou, Ze Li, Jia Shi, and Jun Yu. 2022. ET-BERT: A Contextualized Datagram Representation with Pre-training Transformers for Encrypted Traffic Classification. In *Proceedings of the ACM Web Conference (WWW)*. ET-BERT placeholder – replace before submission..
- [10] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. 2017. iCaRL: Incremental Classifier and Representation Learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Placeholder entry – replace before submission..
- [11] Yue Wu, Yinpeng Chen, Lijuan Wang, Yuanqi Ye, Zicheng Liu, Yandong Guo, and Yun Fu. 2019. Large Scale Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. BiC placeholder – replace before submission..
- [12] Shipeng Yan, Jiangwei Xie, and Xuming He. 2021. DER: Dynamically Expandable Representation for Class Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Placeholder entry – replace before submission..